91.4
Pass

Chellarao Madine

PDF generated at: 12 Feb 2026 06:22:20 UTC

View this report on HackerRank

Score

91.4% • 160.0 / 175
scored in eClerx - Java Coding Test - 3+ Years New in 29 min 28 sec on 12 Feb 2026 11:20:51 IST

Candidate Information

Email	chellaraomadine@gmail.com
Test	eClerx - Java Coding Test - 3+ Years New
Candidate Packet	View
Taken on	12 Feb 2026 11:20:51 IST
Time taken	29 min 28 sec/ 90 min
Work Experience	3 years
Invited by	Nancy
Invited on	12 Feb 2026 10:21:10 IST

Skill Distribution

No.	Skill	Score
1	Spring Boot Advanced	50%

2	Spring Boot Intermediate	0%
3	Problem Solving Intermediate	100%
4	Java Intermediate	100%

Tags Distribution

Spring Boot	40%	Hard	50%
TDD	0%	Spring	0%
Application Properties Annotations	0%	Medium	96%
Sorting	100%	Greedy Algorithms	100%
Two Pointers	100%	Java	100%
Multithreading	100%	Interfaces	100%
OOP	100%		

Questions

Spring Boot (Advanced) • 10 / 25

No.	Question	Skill	Score	Code Quality	Time Taken	Status

1	JSON Deserialization Multiple Choice	Spring Boot (Advanced)	0/5	-	1 min 10 sec	✗
2	Cracker Multiple Choice	Spring Boot (Advanced)	5/5	-	51 sec	✓
3	Determine the Output Multiple Choice	Spring Boot (Advanced)	5/5	-	1 min 54 sec	✓
4	Output Multiple Choice	Spring Boot (Advanced)	0/5	-	36 sec	✗
5	Configuration Properties Multiple Choice	Spring Boot (Intermediate)	0/5	-	8 sec	✗

Problem Solving (Intermediate) • 75 / 75

No.	Question	Skill	Score	Code Quality	Time Taken	Status
6	Maximum Array Correlation Coding	Problem Solving (Intermediate)	75/75	-	7 min 13 sec	✓

Java (Intermediate) • 75 / 75

No.	Question	Skill	Score	Code Quality	Time Taken	Status

7	Java: Safe Cracking Coding	Java (Intermediate)	75/75	-	17 min 9 sec	✓

Candidate Feedback

great

1. JSON Deserialization

Incorrect

Multiple Choice | Spring Boot | Hard

Question description

CheckerCreateRequest.java

```
// Annotation here
public class CheckerCreateRequest {
   private String name;
   private String surname;
   public String getName() {
      return name;
   }
   public void setName(String name) {
      this.name = name;
   }
   public String getSurname() {
      return surname;
   }
   public void setSurname(String surname) {
      this.surname = surname;
   }
}
```

CheckerController.java

```
@RestController
@RequestMapping("/checkers")
public class CheckerController {
```

```
    @Autowired
    CheckerService checkerService;

    @PostMapping
    public ResponseEntity createChecker(@RequestBody CheckerCreateRequest checkerCreateReque
st) {
        checkerService.createChecker(checkerCreateRequest);
        return new ResponseEntity(HttpStatus.CREATED);
    }
}
```

Request Json data

```
{
    "checker":{
        "name": "Checker",
        "surname":"Number2"
    }
}
```

A POST request with "Request Json data" is made to "/checkers". What annotation needs to be added to the "CheckerCreateRequest" class so that it can successfully deserialize and process the request?

Interviewer guidelines

@JsonRootName doesn't add checker as a wrapper attribute by itself. To make it successfully deserialize with given json annotations on D should be used together.

Candidate's Solution

Options: (Expected answer indicated with a tick)

- ● **@JsonRootName("checker")**
- ○ @JsonName("checker")

@JsonTypeName(“checker”)

@JsonTypeInfo(include = JsonTypeInfo.As.WRAPPER_OBJECT, use = JsonTypeInfo.Id.NAME)
@JsonTypeName("checker")

No comments.

2. Cracker

Correct

Multiple Choice | Spring Boot | Hard

Question description

Cracker.java

```
@Component
public class Cracker {
   public void startCracking(){
      System.out.println("Cracking...");
   }
   public void stopCracking(){
      System.out.println("Cracking Stopped!");
   }
}
```

CrackerService.java

```
@Service
public class CrackerService {

   @Autowired
   Cracker cracker;
   public void workWithNewCracker() {
      cracker.stopCracking();
```

```
        cracker = new Cracker();
    }
    public void startWorking() {
        cracker.startCracking();
    }
}
```

CrackerWorker.java

```
@Service
public class CrackerWorker {

    @Autowired
    CrackerService crackerService;

    public void startWorking() {
        crackerService.startWorking();
        crackerService.workWithNewCracker();
        crackerService = new CrackerService();
        crackerService.startWorking();
    }
}
```

Consider these classes are managed in Spring Boot. What happens when 'crackerWorker.startWorking()' is called in the Spring context?

Interviewer guidelines

CrackerService in CrackerWorker will work perfectly until it's replaced with new CrackerService. When that happens it just replace existing injected bean with empty pojo. Which doesn't have it's cracker object so when it's startWorking method is called it will try to execute null cracker attributes startWorking method and NullPointerException will arise.

Candidate's Solution

Options: (Expected answer indicated with a tick)

- ○ "Cracking..." "Cracking Stopped!" will be written to console

"Cracking..." "Cracking Stopped!" "Cracking..." will be written to console

"Cracking..." "Cracking Stopped!" NullPointerException will be thrown

None of above

No comments.

3. Determine the Output

Correct

Multiple Choice | Spring Boot | Hard

Question description

SpringQuestionApplication.java

```
package com.hackerrank.spring;
import org.springframework.boot.SpringApplication;
import org.springframework.boot.autoconfigure.SpringBootApplication;
@SpringBootApplication
public class SpringQuestionsApplication {
   public static void main(String[] args) {
      SpringApplication.run(SpringQuestionsApplication.class, args);
   }
}
```

Lie.java

```
package com.hackerrank;
import org.springframework.boot.CommandLineRunner;
```

```
import org.springframework.stereotype.Component;
@Component
public class Lie implements CommandLineRunner {
    @Override
    public void run(String... args) throws Exception {
        System.out.println("Truth is behind the lies");
        System.out.println("This is working perfectly");
        throw new Exception("Does this work?");
    }
}
```

Truth.java

```
package com.hackerrank.spring.reality;

import org.springframework.boot.CommandLineRunner;
import org.springframework.stereotype.Component;

@Component
public class Truth implements CommandLineRunner {
    @Override
    public void run(String... args) throws Exception {
        System.out.println("Truth is in your eyes");
    }
}
```

What will be printed on the console when the "SpringBootApplication" class is executed?

Interviewer guidelines

Correct Answer:

Option [A] – "Truth is in your eyes"

Explanation: Spring Boot scans only the package of the `@SpringBootApplication` class and its subpackages. `Truth` is under a subpackage and runs. `Lie` is in a parent package, so it is **not scanned** and its runner does not execute.

Incorrect Answers:

Option [B] – Multiple prints + exceptions

Explanation: `Lie` never runs, so its prints and exception do not occur.

Option [C] – Prints from Lie only

Explanation: `Truth` is discovered and executed; `Lie` is not.

Option [D] – Prints from Lie, then Truth
Explanation: `Lie` is outside the scan path and never executed.

Candidate's Solution

Options: (Expected answer indicated with a tick)

- [x] Truth is in your eyes
- [ ] Truth is in your eyes Truth is behind the lies This is working perfectly And exception occurs and application stops
- [ ] Truth is behind the lies This is working perfectly And exception occurs and application stops
- [ ] Truth is behind the lies This is working perfectly And exception occurs containing “Does this work?” Truth is in your eyes

No comments.

---

4. Output

Incorrect

Multiple Choice | Hard | Spring Boot | TDD

Question description

What will happen to the output file?

```
@SpringBootTest
public class LogTest
{
   File log;
   FileWriter logWriter;

   @Autowired
   SomeClassToTest someClassToTest;

   @Before
   public void init()
   {
     log = new File("log.txt");
     logWriter = new FileWriter(log);
   }

   @After
   public void teardown()
   {
      log.delete();
   }

   @Test
   public void test1()
   {
      logWriter.write("Entered Test 1");
      assertTrue(someClassToTest.someMethod());
   }

   @Test
   public void test2()
   {
      logWriter.write("Entered Test 2");
      assertTrue(someClassToTest.otherMethod());
   }
}
```

Candidate's Solution

Options: (Expected answer indicated with a tick)

- The output file is not created.
- **The output file is created but not deleted.**
- The output file is created and eventually deleted.
- The create and delete blocks will not be executed.

No comments.

5. Configuration Properties

Incorrect

Multiple Choice | Spring | Spring Boot | Application Properties Annotations | Medium

Question description

An *application.properties* file contains the following properties:

```
app.prop1 = value1
app.prop2 = value2
```

You decided to create a class to hold this configuration. To protect it, the class is defined as final.

```
@Configuration
@ConfigurationProperties(prefix = "app")
public final class AppConfiguration {
  String prop1;
```

```
  String prop2;

  public String getProp1() {
    return prop1;
  }

  public void setProp1(String prop1) {
    this.prop1 = prop1;
  }

  public String getProp2() {
    return prop2;
  }

  public void setProp2(String prop2) {
    this.prop2 = prop2;
  }
}
```

What is the result of running the Spring application with this class?

Interviewer guidelines

Spring creates dynamic proxies for classes annotated with `@Configuration` classes. Spring uses CGLIB to extend your class to create proxy. Hence, configuration classes cannot be final. When running the code Spring throws `BeanDefinitionParsingException` and this is the correct option.

Candidate's Solution

Options: (Expected answer indicated with a tick)

- ○ The application runs, the configuration bean is created. and the properties are initialized correctly (prop1=value1, prop2=value2).
- ● ** The application runs, and a configuration bean is created, but properties are set to null (prop1=null, prop2=null). **

- [ ] A BeanDefinitionParsingException is thrown because the @Configuration bean cannot be marked as final.

- [ ] An IllegalStateException is thrown because the @Configuration bean cannot have an @ConfigurationProperties annotation.

- [ ] A NoSuchMethodException is thrown because the @Configuration bean needs a default constructor.

No comments.

6. Maximum Array Correlation

Correct

Coding | Sorting | Greedy Algorithms | Medium | Two Pointers

Question description

For two arrays *a* and *b* of equal length, the array correlation is defined as the sum of all values *b[i]* where *b[i]* is greater than *a[i]*.

Given two integer arrays *a* and *b* of the same length, rearrange the elements of array *b* to maximize the array correlation. Return the maximum possible array correlation value.

Note: You are not allowed to rearrange the elements of the array *a*.

Example:
n = 5, *a* = [1, 2, 3, 4, 5], *b* = [3, 5, 4, 6, 2]

b can be rearranged to [2, 3, 4, 5, 6]. Now, for each index, the integers in *b* are greater than those

in *a*. The array correlation is 2 + 3 + 4 + 5 + 6 = 20.

Function Description

Complete the function *getMaxArrayCorrelation* in the editor with the following parameter(s):

a[n]: the fixed order array

b[n]: the array to reorder

Returns:

long int: the maximum possible array correlation

Constraints

- $1 \le n \le 2 \times 10^5$
- $1 \le a[i], b[i] \le 10^9$

▼ INPUT FORMAT FOR CUSTOM TESTING

The first line contains an integer, *n*, that represents the number of elements in *a*.

Each line *i* of the *n* subsequent lines (where $0 \le i < n$) contains an integer that represents *a[i]*.
The next line contains an integer, *n*, that represents the number of elements in *b*.
Each line *i* of the *n* subsequent lines (where $0 \le i < n$) contains an integer that represents *b[i]*.

▼ SAMPLE CASE 0

Sample Input For Custom Testing

```
STDIN       FUNCTION
-------     -----------
5       →  a[] size n = 5
1       →  a = [1, 4, 2, 1, 3]
4
2
1
3
5       →  b[] size n = 5
2       →  b = [2, 3, 1, 2, 2]
3
1
2
2
```

Sample Output

```
7
```

Explanation

The second array can be rearranged to *b = [2, 1, 3, 2, 2]*. *b[i] > a[i]* for *i =* 0, 3, and 4, and the array correlation is 2 + 3 + 2 = 7.

▼ SAMPLE CASE 1

Sample Input For Custom Testing

```
STDIN       FUNCTION
-------     ----------
4       →  a[] size n = 4
1       →  a = [1, 9, 4, 2]
9
4
2
4       →  b[] size n = 4
8       →  b = [8, 4, 3, 1]
4
3
1
```

Sample Output

```
15
```

Explanation

The second array can be rearranged to *b = [3, 1, 8, 4]*. *b[i] > a[i]* for *i =* 0, 3, and, 4. The array correlation is 3 + 8 + 4 = 15.

Interviewer guidelines

▼ SOLUTION

Skills: Greedy Algorithms, Sorting

Optimal Solution:

We can use a greedy approach here. We will always try to take the maximum values of b as much as possible. We can sort the two arrays in reverse order and iterate through the array *a.* We add *b[j]* to the score and increment *j* at each element where *a[i] < b[j].*

Python Solution:

```
def getMaxArrayCorrelation(a, b):
    list.sort(a, reverse = True)
    list.sort(b, reverse = True)
    ans = 0
    i, j = 0, 0
    n = len(a)
    while i < n and j < n:
        if a[i] < b[j]:
            ans += b[j]
            j += 1
        i += 1
    return ans
```

▼ COMPLEXITY ANALYSIS

Time Complexity - O(n logn)

Sorting the arrays takes O(n logn) time in the worst case.

Space Complexity - O(n)

Assuming that the inbuilt sorting functions use a variation of merge sort, sorting uses O(n) extra space.

Candidate's Solution

Language used: **Python 3**

```
#!/bin/python3

import math
import os
import random
import re
import sys


#
# Complete the 'getMaxArrayCorrelation' function below.
#
# The function is expected to return a LONG_INTEGER.
# The function accepts following parameters:
#  1. INTEGER_ARRAY a
```

```python
#  2. INTEGER_ARRAY b
#

def getMaxArrayCorrelation(a, b):
    a_sort = sorted(a, reverse=True)
    b_sort = sorted(b, reverse=True)
    n=len(a)
    total =0
    a_ptr=0
    b_ptr=0
    while a_ptr<n and b_ptr<n:
        if b_sort[b_ptr]>a_sort[a_ptr]:
            total += b_sort[b_ptr]
            b_ptr +=1
            a_ptr +=1
        else:
            a_ptr +=1

    return total

if __name__ == '__main__':
    fptr = open(os.environ['OUTPUT_PATH'], 'w')

    a_count = int(input().strip())

    a = []

    for _ in range(a_count):
        a_item = int(input().strip())
        a.append(a_item)

    b_count = int(input().strip())

    b = []

    for _ in range(b_count):
        b_item = int(input().strip())
        b.append(b_item)

    result = getMaxArrayCorrelation(a, b)

    fptr.write(str(result) + '\n')

    fptr.close()

```

TESTCASE	DIFFICULTY	TYPE	STATUS	SCORE	TIME TAKEN	MEMORY USED
Testcase 0	Easy	Sample	Success	1	0.0254 sec	10.6 KB
Testcase 1	Easy	Sample	Success	1	0.0264 sec	10.6 KB
Testcase 2	Easy	Sample	Success	1	0.025 sec	10.6 KB
Testcase 3	Easy	Hidden	Success	3	0.0284 sec	10.6 KB
Testcase 4	Easy	Hidden	Success	3	0.0286 sec	10.6 KB
Testcase 5	Medium	Hidden	Success	4	0.0244 sec	10.6 KB
Testcase 6	Medium	Hidden	Success	5	0.0254 sec	10.6 KB
Testcase 7	Medium	Hidden	Success	6	0.0322 sec	10.8 KB
Testcase 8	Medium	Hidden	Success	6	0.034 sec	10.8 KB
Testcase 9	Hard	Hidden	Success	7	0.0561 sec	11.6 KB
Testcase 10	Hard	Hidden	Success	7	0.1091 sec	13.1 KB
Testcase 11	Hard	Hidden	Success	7	0.1777 sec	15.1 KB

Testcase 12	Hard	Hidden	Success	7	0.2224 sec	17.1 KB
Testcase 13	Hard	Hidden	Success	8	0.6289 sec	29.5 KB
Testcase 14	Hard	Hidden	Success	9	0.6327 sec	29 KB

No comments.

7. Java: Safe Cracking

Correct

Coding Medium Java Multithreading Interfaces OOP

Question description

Implement a *SafeCracking* class that executes a sequence of operations to rob a bank vault. The class should coordinate with an *IActions* interface to perform the robbery in the correct order.

Your implementation must follow a specific sequence of operations:

1. First, unlock the vault door.
2. Then, simultaneously determine the safe deposit box number and hack its PIN.
3. Finally, open the safe deposit box using the obtained information.

Method to Implement

```
void rob(IActions actions)
```

This method should coordinate with the provided *IActions* object to execute the following steps in order:

1. Call *unlockTheDoor()* to get the door passcode string.
2. Concurrently call both:
   - *hackSecretPin(passcodeOfDoor)* to get the safe deposit box PIN
   - *figureOutSafeDepositBoxNumber(passcodeOfDoor)* to get the safe deposit box number

3. Call *openSafeDepositLock(safeDepositBoxNumber, safetyDepositBoxPin)* to complete the robbery.

Note: The concurrent operations in step 2 are critical for the time-sensitive nature of the robbery scenario.

Example

The system will validate your implementation by checking if the operations are called in the correct order with the proper parameters. A successful implementation will output:

```
true
```

This indicates that all operations were performed correctly, and the robbery was successful. If the operations are performed in the wrong order or with incorrect parameters, the output will be *false*.

Constraints

- *1* ≤ lengths of *passcodeOfDoor*, *safeDespositBoxNumber, safetyDepositBoxPin* ≤ *10*

▼ INPUT FORMAT FOR CUSTOM TESTING

There is no input passed from stdin.

▼ SAMPLE CASE 0

Sample Output

```
true
```

Explanation

- stdout shows *true* if it is a successful robbery otherwise, it shows *false.*

Interviewer guidelines

Java 8 and Java 15 Solution

```
class SafeCracking {
   public void rob(IActions actions) throws InterruptedException, ExecutionException{
      ExecutorService threadPool = Executors.newFixedThreadPool(2);
      String passcodeOfDoor = actions.unlockTheDoor();
      Future<String> hackSecretPinTask = threadPool.submit(() -> actions.hackSecretPin(passcodeOf
Door));
      Future<String> figureOutSafeDepositBoxNumberTask = threadPool.submit(() -> actions.figureO
utSafeDepositBoxNumber(passcodeOfDoor));
```

```
        String safeDespositBoxNumber = figureOutSafeDepositBoxNumberTask.get();
        String safeDepositBoxPin = hackSecretPinTask.get();

        actions.openSafeDepositLock(safeDespositBoxNumber, safeDepositBoxPin);
        threadPool.shutdown();
    }
}
```

Candidate's Solution

Language used: **Java 21**

```java
import java.io.*;
import java.util.*;
import java.util.concurrent.ExecutionException;
import java.util.concurrent.ExecutorService;
import java.util.concurrent.Executors;
import java.util.concurrent.Future;

interface IActions{
    public String unlockTheDoor();
    public String hackSecretPin(final String passcodeOfDoor) throws InterruptedException, ExecutionException;
    public String figureOutSafeDepositBoxNumber(final String passcodeOfDoor) throws InterruptedException, ExecutionException;
    public void openSafeDepositLock(final String safetyDespositBoxNumber, final String safetyDepositBoxPin);
    public boolean getRobberyVerdict();
}

import java.io.*;
import java.util.*;
import java.util.concurrent.CompletableFuture;
import java.util.concurrent.CompletionException;
import java.util.concurrent.ExecutionException;
import java.util.concurrent.ExecutorService;
import java.util.concurrent.Executors;
import java.util.concurrent.Future;
import org.apache.hc.core5.concurrent.CompletedFuture;

interface IActions{
```

```
    public String unlockTheDoor();
    public String hackSecretPin(final String passcodeOfDoor) throws InterruptedException, ExecutionException;
    public String figureOutSafeDepositBoxNumber(final String passcodeOfDoor) throws InterruptedException, ExecutionException;
    public void openSafeDepositLock(final String safetyDespositBoxNumber, final String safetyDepositBoxPin);
    public boolean getRobberyVerdict();
}


class SafeCracking {
    public void rob(IActions actions) throws InterruptedException, ExecutionException{
        String passcode = actions.unlockTheDoor();
        CompletableFuture<String> pinFuture = CompletableFuture.supplyAsync(() -> {
            try{
                return actions.hackSecretPin(passcode);
            }catch(Exception e){
                throw new CompletionException(e);
            }

        });
         CompletableFuture<String> boxFuture = CompletableFuture.supplyAsync(() -> {
             try{
                return actions.figureOutSafeDepositBoxNumber(passcode);
            }catch(Exception e){
                throw new CompletionException(e);
            }

        });
        CompletableFuture.allOf(pinFuture,boxFuture).join();

        String pin = pinFuture.get();
        String boxNumber = boxFuture.get();

        actions.openSafeDepositLock(boxNumber, pin);

    }
}

public class Solution {
    public static void main(String[] args) throws InterruptedException, ExecutionException{
```

```
        IActions actions = Actions.actions;
        new SafeCracking().rob(actions);
        System.out.println(actions.getRobberyVerdict());
    }
}

class Actions implements IActions{
    public static IActions actions = new Actions();

    private String passcodeOfDoor;
    private String safetyBoxNumber;
    private String pinOfSafetyBox;

    private String passedPasscodeOfDoor;
    private String passedSafetyBoxNumber;
    private String passedPinOfSafetyBox;

    private Actions() {
        this.passcodeOfDoor = randomStringGenerator(10);
        this.safetyBoxNumber = randomStringGenerator(10);
        this.pinOfSafetyBox = randomStringGenerator(10);
    }

    private String randomStringGenerator(int targetStringLength) {
        int leftLimit = 48; // numeral '0'
        int rightLimit = 122; // letter 'z'
        Random random = new Random();

        String generatedString = random.ints(leftLimit, rightLimit + 1)
            .filter(i -> (i <= 57 || i >= 65) && (i <= 90 || i >= 97))
            .limit(targetStringLength)
            .collect(StringBuilder::new, StringBuilder::appendCodePoint, StringBuilder::append)
            .toString();

        return generatedString;
    }

    public boolean getRobberyVerdict(){
        return (passcodeOfDoor == passedPasscodeOfDoor) && (pinOfSafetyBox == passedPinOfSafetyBox)
            && (safetyBoxNumber == passedSafetyBoxNumber);
    }

    public String unlockTheDoor() {
        return this.passcodeOfDoor;
```

```
    }

    public String hackSecretPin(final String passcodeOfDoor) throws InterruptedException, ExecutionException {
        this.passedPasscodeOfDoor = passcodeOfDoor;
        Thread.sleep(2000);
        return this.pinOfSafetyBox;
    }

    public  String figureOutSafeDepositBoxNumber(final String passcodeOfDoor) throws InterruptedException, ExecutionException {
        Thread.sleep(2000);
        return this.safetyBoxNumber;
    }

    public void openSafeDepositLock(final String safetyDespositBoxNumber, final String safetyDepositBoxPin) {
        this.passedSafetyBoxNumber = safetyDespositBoxNumber;
        this.passedPinOfSafetyBox = safetyDepositBoxPin;
    }
}
public class Solution {
    public static void main(String[] args) throws InterruptedException, ExecutionException{
        IActions actions = Actions.actions;
        new SafeCracking().rob(actions);
        System.out.println(actions.getRobberyVerdict());
    }
}

class Actions implements IActions{
    public static IActions actions = new Actions();

    private String passcodeOfDoor;
    private String safetyBoxNumber;
    private String pinOfSafetyBox;

    private String passedPasscodeOfDoor;
    private String passedSafetyBoxNumber;
    private String passedPinOfSafetyBox;

    private Actions() {
        this.passcodeOfDoor = randomStringGenerator(10);
        this.safetyBoxNumber = randomStringGenerator(10);
        this.pinOfSafetyBox = randomStringGenerator(10);
    }
```

```java

    private String randomStringGenerator(int targetStringLength) {
        int leftLimit = 48; // numeral '0'
        int rightLimit = 122; // letter 'z'
        Random random = new Random();

        String generatedString = random.ints(leftLimit, rightLimit + 1)
            .filter(i -> (i <= 57 || i >= 65) && (i <= 90 || i >= 97))
            .limit(targetStringLength)
            .collect(StringBuilder::new, StringBuilder::appendCodePoint, StringBuilder::append)
            .toString();

        return generatedString;
    }

    public boolean getRobberyVerdict(){
        return (passcodeOfDoor == passedPasscodeOfDoor) && (pinOfSafetyBox == passedPinOfSafetyBox)
            && (safetyBoxNumber == passedSafetyBoxNumber);
    }

    public String unlockTheDoor() {
        return this.passcodeOfDoor;
    }

    public String hackSecretPin(final String passcodeOfDoor) throws InterruptedException, ExecutionException {
        this.passedPasscodeOfDoor = passcodeOfDoor;
        Thread.sleep(2000);
        return this.pinOfSafetyBox;
    }

    public  String figureOutSafeDepositBoxNumber(final String passcodeOfDoor) throws InterruptedException, ExecutionException {
        Thread.sleep(2000);
        return this.safetyBoxNumber;
    }

    public void openSafeDepositLock(final String safetyDespositBoxNumber, final String safetyDepositBoxPin) {
        this.passedSafetyBoxNumber = safetyDespositBoxNumber;
        this.passedPinOfSafetyBox = safetyDepositBoxPin;
    }
}
```

TESTCASE	DIFFICULTY	TYPE	STATUS	SCORE	TIME TAKEN	MEMORY USED
Testcase 0	Easy	Sample	Success	1	2.0575 sec	38 KB
Testcase 1	Hard	Hidden	Success	74	2.0526 sec	37.6 KB

No comments.